CS4031 – Compiler Construction
Assignment 3
Report

Imama Sarwar (23I-3048)
Menahil Ahmad (23I-0546)
CS-D

1. Introduction

Overview of Bottom-Up Parsing

Bottom-up parsing, also known as shift-reduce parsing, is a powerful parsing technique that constructs a parse tree by starting from the input tokens (leaves) and working backwards toward the start symbol (root). Unlike top-down parsers that predict what production to use, bottom-up parsers shift input symbols onto a stack and reduce them according to grammar productions when the right-hand side matches the top of the stack.

LR Parsers

LR parsers represent the most general class of deterministic bottom-up parsers. The name "LR" stands for:

- L: Left-to-right scanning of input
- R: Rightmost derivation in reverse

The LR parser family includes:

- LR(0): Simplest but least powerful; uses no lookahead
- SLR(1): Simple LR with 1 token lookahead; uses FOLLOW sets for reductions
- LR(1): Canonical LR with 1 token lookahead; uses item-specific lookaheads
- LALR(1): Lookahead LR; merges LR(1) states with identical cores

Why LR Parsers?

LR parsers offer several advantages:

1. Wide language coverage: Can parse virtually all programming language constructs
2. Early error detection: Detect syntax errors as soon as possible
3. Efficiency: Linear time complexity O(n) where n is input length
4. Deterministic: No backtracking required

Project Objectives

This project implements both SLR(1) and LR(1) parsers with the following capabilities:

- Parse context-free grammars from file input
- Construct LR(0) and LR(1) item canonical collections
- Build SLR(1) and LR(1) parsing tables

- Handle shift/reduce and reduce/reduce conflicts
- Parse input strings and generate parse trees
- Compare the power and state complexity of SLR(1) vs LR(1)

2. Approach

Data Structures

- Items:
```
public class LRItem {
    public final String lhs;              // Left-hand side non-terminal
    public final List<String> rhs;        // Right-hand side symbols
    public final int dotPosition;         // Position of the dot in the production
    public final Set<String> lookaheads;  // Lookahead set (for LR(1))
}
```
- Item Sets
  `Set<LRItem>`, to prevent duplicate items in a state
- Parsing Tables
  - The parsing table class uses 2 tables:
    ```
    private final Map<Integer, Map<String, Action>> actionTable;
    private final Map<Integer, Map<String, Integer>> gotoTable;
    ```
  - For a table with conflicts there is also
    ```
    private final Map<Integer, Map<String, List<Action>>> conflictTable;
    private final List<Conflict> conflicts;
    ```

Algorithm Implementation Details

SLR(1)

CLOSURE for SLR(1) takes a set of LR(0) items and adds all items that can be reached by expanding non-terminals that appear immediately after the dot. For any item of the form [A → α • B β] where B is a non-terminal, CLOSURE adds every production of B with the dot at the beginning [B → • γ]. This process repeats until no new items can be added. The result represents all possible parsing configurations reachable from the current state without consuming any input.

GOTO for SLR(1) computes the transition from a state I on a grammar symbol X. It first finds all items in I where the dot is immediately before X, then shifts the dot past X to create new items [A → α X • β]. Finally, it applies CLOSURE to this set of shifted items. The result becomes a new DFA state. This operation forms the transitions in the LR automaton and is used to build both the ACTION table (for terminals) and GOTO table (for non-terminals).

```
Algorithm CLOSURE(I):
    closure = I
    repeat until no change:
        for each item [A → α • B β] in closure:
            for each production B → γ:
                add [B → • γ] to closure
```

```
    return closure

Algorithm GOTO(I, X):
    shifted = empty set
    for each item [A → α • X β] in I:
        add [A → α X • β] to shifted
    return CLOSURE(shifted)
```

LR(1)

CLOSURE for LR(1) extends the SLR(1) version by incorporating lookahead information. For an item [A → α • B β, a], CLOSURE adds [B → • γ, b] for every production B → γ, where b is any terminal in FIRST(βa). The lookaheads are computed from the first set of the symbols following B plus the original lookahead a. If there is no lookahead symbol, the lookahead is inherited. This ensures that each item carries precise lookahead information about which terminals can legally follow the completed production.

GOTO for LR(1) works identically to SLR(1) but preserves lookaheads during the dot shift. When moving the dot past symbol X from [A → α • X β, a], the resulting item is [A → α X • β, a] with the same lookahead a. After collecting all shifted items, GOTO applies CLOSURE_LR1 to propagate lookaheads appropriately. This lookahead propagation is what gives LR(1) its greater parsing power compared to SLR(1), allowing it to resolve conflicts that SLR(1) cannot.

```
Algorithm CLOSURE_LR1(I):
    closure = I
    repeat until no change:
        for each item [A → α • B β, a] in closure:
            for each production B → γ:
                for each terminal b in FIRST(βa):
                    add [B → • γ, b] to closure
    return closure

Algorithm GOTO_LR1(I, X):
    Input:  I = set of LR(1) items
            X = grammar symbol (terminal or non-terminal)
    Output: J = GOTO(I, X)

    J = empty set

    for each item [A → α • X β, a] in I:
        shiftedItem = [A → α X • β, a]      // Move dot past X, preserve lookahead
        add shiftedItem to J

    return CLOSURE_LR1(J)
```

Design decisions and trade-offs

- Single LRItem class for both SLR(1) and LR(1): Reusing one class with an optional lookahead set simplifies code but wastes memory for SLR(1) where lookaheads remain empty.

- Queue-based CLOSURE over recursion: Iterative approach prevents stack overflow on large grammars and is easier to debug, though slightly slower than recursion.
- Separate GOTO methods for SLR(1) and LR(1): Duplicates logic but keeps algorithms clear and maintainable. The alternative would be a unified method with flags, reducing code clarity.
- Storing conflict actions separately from table: Allows conflict resolution (e.g., shift over reduce for dangling-else) without losing original conflict information for reporting.
- Computing FIRST(βa) per lookahead individually in LR(1): Correct but expensive. Memoization could improve performance at the cost of additional memory.

How Lookaheads were Handled in LR(1)

When computing CLOSURE for LR(1), for an item `[A → α • B β, a]`, we add `[B → • γ, b]` for each production `B → γ`, where `b` is any terminal in FIRST(βa). If β is empty or all symbols in β are nullable, then FIRST(βa) contains the original lookahead `a`, which gets **inherited** to the new item. This inheritance ensures lookaheads propagate through nullable non-terminals. During GOTO, lookaheads are preserved unchanged when shifting the dot. For reduce actions, LR(1) uses the item's own lookahead set rather than FOLLOW sets, placing reduce actions only for those specific terminals. This precise lookahead propagation distinguishes LR(1) from SLR(1).

LR(1) handles shift reduce conflicts by using disambiguating rule: prefer shift over reduce

3. Challenges and Solutions

Challenge 1: FIRST and FOLLOW Set Computation

Difficulty: Computing FIRST and FOLLOW sets correctly, especially for grammars with epsilon productions and nullable non-terminals.

Solution: Implemented iterative fixed-point algorithms that repeatedly process productions until no changes occur. For FIRST sets, we propagate terminals through productions and handle epsilon carefully. For FOLLOW sets, we added the end-of-input marker $ to the start symbol and propagated FOLLOW information through nullable symbols.

Challenge 2: LR(1) Item Closure with Lookaheads

Difficulty: The LR(1) closure operation requires computing FIRST sets of strings that include lookaheads, which is significantly more complex than LR(0) closure.

Solution: Implemented a specialized closureLR1() function that:

1. For each item [A -> α • B β, a] where B is non-terminal

2. For each production B -> γ
3. For each terminal b in FIRST(βa)
4. Add [B -> • γ, b] to the closure

The key insight was creating the firstOfSequence() helper that computes FIRST for any string of symbols including the lookahead.

Challenge 3: Conflict Detection and Resolution

Difficulty: Detecting and reporting shift/reduce and reduce/reduce conflicts accurately.

Solution: Implemented conflict detection in the parsing table construction:

- Shift/Reduce: Occurs when a state has both a shift and reduce action on the same terminal
- Reduce/Reduce: Occurs when a state has two different reduce actions on the same terminal

The ParsingTable class maintains a conflict table and provides detailed conflict reporting, making it easy to identify problematic grammar constructs.

Challenge 4: LR(1) State Explosion

Difficulty: LR(1) parsers can generate significantly more states than SLR(1) parsers, making debugging difficult.

Solution: Implemented comprehensive state printing with lookahead information displayed clearly. For our test grammar:

- SLR(1): 12 states
- LR(1): 22 states (83% increase)

The LR(1) items show lookaheads in brackets [lookahead], making it easy to see why states are split based on different lookahead contexts.

Challenge 5: Parse Tree Construction

Difficulty: Building a useful parse tree during shift-reduce parsing that can be visualized.

Solution: Created a Tree class that builds nodes during reduce actions:

- Each reduction creates a new node labeled with the production's LHS
- Child nodes are the symbols from the production's RHS
- The tree is built bottom-up as reductions occur

For visualization, we output Graphviz DOT format, which can be rendered to images using external tools.

Challenge 6: Grammar File Parsing

Difficulty: Supporting multi-character non-terminals (e.g., Expr, Term, Factor) while distinguishing terminals (lowercase).

Solution: Implemented validation rules:

- Non-terminals must start with uppercase letters
- Terminals are everything else (lowercase letters, operators, parentheses)
- Epsilon represented by @ symbol

Challenge 7: Table-Driven Parsing Algorithm

Difficulty: Implementing the shift-reduce algorithm correctly with stack operations and action lookup.

Solution: Implemented a clear algorithm:

1. Initialize stack with state 0
2. Lookup action for current state and input symbol
3. For shift: push symbol and new state, advance input
4. For reduce: pop $2\times|\beta|$ items, lookup goto, push LHS and new state
5. For accept: parsing successful

The parsing trace shows each step with stack contents, remaining input, and action taken.

Challenge 8: Code Reusability Between SLR(1) and LR(1)

Difficulty: SLR(1) and LR(1) share 80% of the implementation but differ in key areas (closure, GOTO, reduce actions).

Solution: Designed the code with reusability in mind:

- Items class has both closure() (SLR) and closureLR1() (LR1)
- ParsingTable uses a useLookaheads flag to switch between algorithms
- Same SLRParser class works with both table types
- Stack and Tree classes are completely reusable

4. Test Cases

Grammar 2

```
Expr -> Expr + Term | Term
Term -> Term * Factor | Factor
Factor -> ( Expr ) | id
```

Valid:

```
id + id
id + id * id
( id + id ) * id
```

Invalid:

```
id + * id
( id + id
```

Grammar 3

```
Start -> L = R | R
L -> * R | id
R -> L
```

Valid:

```
* id
id = id
* * id = id
```

Invalid:

```
id = = id
* = id
```

Grammar 4

```
Stmt -> if Expr then Stmt | if Expr then Stmt else Stmt | other
Expr -> id
```

Valid:

```
if id then other
if id then other else other
if id then if id then other else other
```

Invalid:

```
if id then
if id then other else
```

Grammars 3 and 4 demonstrate test cases where conflict is resolved by LR(1)

Grammar 3

SLR(1) Parsing Table:

State	$	*	=	id	L	R	Start
0	-	s3	-	s4	5	1	2
1	r(Start -> R)	-	-	-	-	-	-
2	accept	-	-	-	-	-	-
3	-	s3	-	s4	7	6	-
4	r(L -> id)	-	r(L -> id)	-	-	-	-
5	r(R -> L)	-	s8 / r(R -> L)	-	-	-	-
6	r(L -> * R)	-	r(L -> * R)	-	-	-	-
7	r(R -> L)	-	r(R -> L)	-	-	-	-
8	-	s3	-	s4	7	9	-
9	r(Start -> L = R)	-	-	-	-	-	-

Parser Type: SLR(1)
Is SLR(1) Parseable: NO
Conflicts Found: 1

--- LR(1) Parsing Table ---

State	$	*	=	id	L	R	Start
0	-	s3	-	s4	5	1	2
1	r(Start -> R)[$]	-	-	-	-	-	-
2	accept	-	-	-	-	-	-
3	-	s3	-	s4	7	6	-
4	r(L -> id)[$]	-	r(L -> id)[=]	-	-	-	-
5	r(R -> L)[$]	-	s8	-	-	-	-
6	r(L -> * R)[$]	-	r(L -> * R)[=]	-	-	-	-
7	r(R -> L)[$]	-	r(R -> L)[=]	-	-	-	-
8	-	s10	-	s11	12	9	-
9	r(Start -> L = R)[$]	-	-	-	-	-	-
10	-	s10	-	s11	12	13	-
11	r(L -> id)[$]	-	-	-	-	-	-
12	r(R -> L)[$]	-	-	-	-	-	-
13	r(L -> * R)[$]	-	-	-	-	-	-

Parser Type: LR(1)
Is LR(1) Parseable: YES
Conflicts Found: 0

Grammar 4

LR(1) handles shift reduce conflicts by using disambiguating rule: prefer shift over reduce

SLR(1) Parsing Table:

State	$	else	id	if	other	then	Expr	Stmt
0	-	-	-	s2	s1	-	-	3
1	r(Stmt -> other)	r(Stmt -> other)	-	-	-	-	-	-
2	-	-	s5	-	-	-	4	-
3	accept	-	-	-	-	-	-	-
4	-	-	-	-	-	s6	-	-
5	-	-	-	-	-	r(Expr -> id)	-	-
6	-	-	-	-	-	-	-	9
7	-	-	-	s2	s1	-	-	7
8	-	s8 / r(Stmt -> if Expr then Stmt)	-	-	-	-	-	-
9	-	-	-	s2	s1	-	-	9
9	r(Stmt -> if Expr then Stmt else Stmt)	r(Stmt -> if Expr then Stmt else Stmt)	-	-	-	-	-	-

Parser Type: SLR(1)
Is SLR(1) Parseable: NO
Conflicts Found: 1

--- LR(1) Parsing Table ---

State	$	else	id	if	other	then	Expr	Stmt
0	-	-	-	s2	s1	-	-	3
1	r(Stmt -> other)[$]	-	-	-	-	-	-	-
2	-	-	s5	-	-	-	4	-
3	accept	-	-	-	-	-	-	-
4	-	-	-	-	-	s6	-	-
5	-	-	-	-	-	r(Expr -> id)[then]	-	-
6	-	-	-	s8	s7	-	-	9
7	r(Stmt -> other)[$]	r(Stmt -> other)[else]	-	-	-	-	-	-
8	-	-	s5	-	-	-	10	-
9	r(Stmt -> if Expr then Stmt)[$]	s11	-	-	-	-	-	-
10	-	-	-	-	-	s12	-	-
11	-	-	-	s2	s1	-	-	13
12	-	-	-	s8	s7	-	-	14
13	r(Stmt -> if Expr then Stmt else Stmt)[$]	-	-	-	-	-	-	-
14	r(Stmt -> if Expr then Stmt)[$]	s15 / r(Stmt -> if Expr then Stmt)[else]	-	-	-	-	-	-
15	-	-	-	s8	s7	-	-	16
16	r(Stmt -> if Expr then Stmt else Stmt)[$]	r(Stmt -> if Expr then Stmt else Stmt)[else]	-	-	-	-	-	-

Parser Type: LR(1)
Is LR(1) Parseable: YES
Conflicts Found: 1

Conflicts
RESOLVED (Shift > Reduce): SHIFT_REDUCE conflict in state 14 on symbol 'else': shift 15 vs r(Stmt -> if Expr then Stmt)[else]
Resolved by Disambiguating Rule (Shift > Reduce): 1
Remaining Conflicts: 0

5. Comparison Analysis

LR(1) is a stronger parser than SLR(1)

However LR(1) generates more states in its canonical collection
For example, for Grammar 3:

- SLR(1) states: 10
- LR(1) states: 17

Performance Comparison

Grammar 3	Grammar 4

SLR(1):
  Construction Time: 1 ms
  Memory Usage: 4 KB
 	
LR(1):
  Construction Time: 1 ms
  Memory Usage: 8 KB
	SLR(1):
	  Construction Time: 1 ms
	  Memory Usage: 3 KB
	LR(1):
	  Construciton Time: 2 ms
	  Memory Usage: 10 KB

6. Sample Outputs

Sample Output: Grammar 2 (SLR parseable)

```
Augmented Grammar
Expr' -> Expr
Expr -> Expr + Term | Term
Term -> Term * Factor | Factor
Factor -> ( Expr ) | id


================================================================================
PART 1: SLR(1) PARSER IMPLEMENTATION
================================================================================

Canonical Collection (DFA States):

I0:
  Factor -> . ( Expr )
  Expr' -> . Expr
  Term -> . Factor
  Expr -> . Expr + Term
  Expr -> . Term
  Term -> . Term * Factor
  Factor -> . id

I1:
  Expr' -> Expr .
  Expr -> Expr . + Term

I2:
  Term -> Factor .

I3:
  Factor -> ( . Expr )
  Factor -> . ( Expr )
  Term -> . Factor
  Expr -> . Expr + Term
  Expr -> . Term
  Term -> . Term * Factor
  Factor -> . id
```

```
I4:
  Term -> Term . * Factor
  Expr -> Term .

I5:
  Factor -> id .

I6:
  Expr -> Expr + . Term
  Factor -> . ( Expr )
  Term -> . Factor
  Term -> . Term * Factor
  Factor -> . id

I7:
  Expr -> Expr . + Term
  Factor -> ( Expr . )

I8:
  Term -> Term * . Factor
  Factor -> . ( Expr )
  Factor -> . id

I9:
  Expr -> Expr + Term .
  Term -> Term . * Factor

I10:
  Factor -> ( Expr ) .

I11:
  Term -> Term * Factor .
```

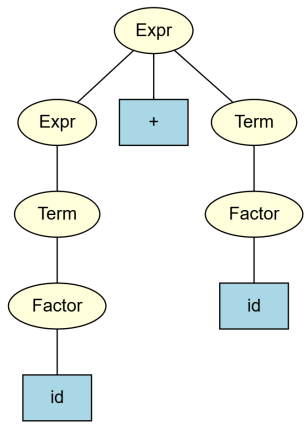
SLR(1) Parsing Table:

State	$	(	)	*	+	id	Expr	Factor	Term
0	-	s3	-	-	-	s5	1	2	4
1	accept	-	-	-	s6	-	-	-	-
2	r(Term -> Factor)	-	r(Term -> Factor)	r(Term -> Factor)	r(Term -> Factor)	-	-	-	-
3	-	s3	-	-	-	s5	7	2	4
4	r(Expr -> Term)	-	r(Expr -> Term)	s8	r(Expr -> Term)	-	-	-	-
5	r(Factor -> id)	-	r(Factor -> id)	r(Factor -> id)	r(Factor -> id)	-	-	-	-
6	-	s3	-	-	-	s5	-	2	9
7	-	-	s10	-	s6	-	-	-	-
8	-	s3	-	-	-	s5	-	11	-
9	r(Expr -> Expr + Term)	-	r(Expr -> Expr + Term)	s8	r(Expr -> Expr + Term)	-	-	-	-
10	r(Factor -> (Expr))	-	r(Factor -> (Expr))	r(Factor -> (Expr))	r(Factor -> (Expr))	-	-	-	-
11	r(Term -> Term * Factor)	-	r(Term -> Term * Factor)	r(Term -> Term * Factor)	r(Term -> Term * Factor)	-	-	-	-

Parser Type: SLR(1)
Is SLR(1) Parseable: YES
Conflicts Found: 0

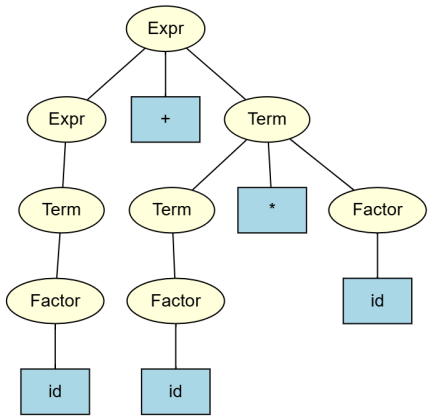
```
Input 1: id + id
SLR(1) Parsing:
SLR Parsing Trace:
Step     Stack                          Input                Action
--------------------------------------------------------------------------------
1        $0id5                          + id $               shift 5
2        $0Factor2                      + id $               reduce Factor -> id
3        $0Term4                        + id $               reduce Term -> Factor
4        $0Expr1                        + id $               reduce Expr -> Term
5        $0Expr1+6                      id $                 shift 6
6        $0Expr1+6id5                   $                    shift 5
7        $0Expr1+6Factor2               $                    reduce Factor -> id
8        $0Expr1+6Term9                 $                    reduce Term -> Factor
9        $0Expr1                        $                    reduce Expr -> Expr + Term
10       $0Expr1                        $                    accept

SLR(1) Parsing result: ACCEPTED
```



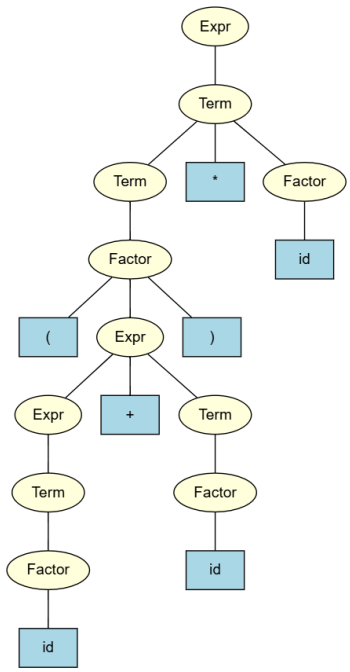

```
Input 2: id + id * id
SLR(1) Parsing:
SLR Parsing Trace:
Step     Stack                          Input                Action
--------------------------------------------------------------------------------
1        $0id5                          + id * id $          shift 5
2        $0Factor2                      + id * id $          reduce Factor -> id
3        $0Term4                        + id * id $          reduce Term -> Factor
4        $0Expr1                        + id * id $          reduce Expr -> Term
5        $0Expr1+6                      id * id $            shift 6
6        $0Expr1+6id5                   * id $               shift 5
7        $0Expr1+6Factor2               * id $               reduce Factor -> id
8        $0Expr1+6Term9                 * id $               reduce Term -> Factor
9        $0Expr1+6Term9*8               id $                 shift 8
10       $0Expr1+6Term9*8id5            $                    shift 5
11       $0Expr1+6Term9*8Factor11       $                    reduce Factor -> id
12       $0Expr1+6Term9                 $                    reduce Term -> Term * Factor
13       $0Expr1                        $                    reduce Expr -> Expr + Term
14       $0Expr1                        $                    accept

SLR(1) Parsing result: ACCEPTED
```




```
Input 3: ( id + id ) * id
SLR(1) Parsing:
SLR Parsing Trace:
Step     Stack                          Input                Action
-------------------------------------------------------------------------------
1        $0(3                           id + id ) * id $     shift 3
2        $0(3id5                        + id ) * id $        shift 5
3        $0(3Factor2                    + id ) * id $        reduce Factor -> id
4        $0(3Term4                      + id ) * id $        reduce Term -> Factor
5        $0(3Expr7                      + id ) * id $        reduce Expr -> Term
6        $0(3Expr7+6                    id ) * id $          shift 6
7        $0(3Expr7+6id5                 ) * id $             shift 5
8        $0(3Expr7+6Factor2             ) * id $             reduce Factor -> id
9        $0(3Expr7+6Term9               ) * id $             reduce Term -> Factor
10       $0(3Expr7                      ) * id $             reduce Expr -> Expr + Term
11       $0(3Expr7)10                   * id $               shift 10
12       $0Factor2                      * id $               reduce Factor -> ( Expr )
13       $0Term4                        * id $               reduce Term -> Factor
14       $0Term4*8                      id $                 shift 8
15       $0Term4*8id5                   $                    shift 5
16       $0Term4*8Factor11              $                    reduce Factor -> id
17       $0Term4                        $                    reduce Term -> Term * Factor
18       $0Expr1                        $                    reduce Expr -> Term
19       $0Expr1                        $                    accept

SLR(1) Parsing result: ACCEPTED
```




```
Input 1: id + * id
SLR(1) Parsing:
SLR Parsing Trace:
Step     Stack                          Input                Action
--------------------------------------------------------------------------------
1        $0id5                          + * id $             shift 5
2        $0Factor2                      + * id $             reduce Factor -> id
3        $0Term4                        + * id $             reduce Term -> Factor
4        $0Expr1                        + * id $             reduce Expr -> Term
5        $0Expr1+6                      * id $               shift 6
6        $0Expr1+6                      * id $               ERROR: no action for (6, *)

SLR(1) Parsing result: REJECTED


Input 2: ( id + id
SLR(1) Parsing:
SLR Parsing Trace:
Step     Stack                          Input                Action
--------------------------------------------------------------------------------
1        $0(3                           id + id $            shift 3
2        $0(3id5                        + id $               shift 5
3        $0(3Factor2                    + id $               reduce Factor -> id
4        $0(3Term4                      + id $               reduce Term -> Factor
5        $0(3Expr7                      + id $               reduce Expr -> Term
6        $0(3Expr7+6                    id $                 shift 6
7        $0(3Expr7+6id5                 $                    shift 5
8        $0(3Expr7+6Factor2             $                    reduce Factor -> id
9        $0(3Expr7+6Term9               $                    reduce Term -> Factor
10       $0(3Expr7                      $                    reduce Expr -> Expr + Term
11       $0(3Expr7                      $                    ERROR: no action for (7, $)

SLR(1) Parsing result: REJECTED
```

Sample Output: Grammar 3 (LR(1) parseable)

```
================================================================================
PART 2: LR(1) PARSER IMPLEMENTATION
================================================================================

--- LR(1) Canonical Collection (with lookaheads) ---

=== LR(1) Item Sets ===

I0:
  L -> . * R, [$]
  L -> . * R, [=]
  L -> . id, [$]
  L -> . id, [=]
  R -> . L, [$]
  Start -> . L = R, [$]
  Start -> . R, [$]
  Start' -> . Start, [$]

I1:
  Start -> R ., [$]

I2:
  Start' -> Start ., [$]

I3:
  L -> . * R, [$]
  L -> . * R, [=]
  L -> . id, [$]
  L -> . id, [=]
  L -> * . R, [$]
  L -> * . R, [=]
  R -> . L, [=]
  R -> . L, [$]

I4:
  L -> id ., [$]
  L -> id ., [=]

I5:
  R -> L ., [$]
  Start -> L . = R, [$]

I6:
  L -> * R ., [$]
  L -> * R ., [=]

I7:
  R -> L ., [=]
  R -> L ., [$]
```

```
I8:
  L -> . * R, [$]
  L -> . id, [$]
  R -> . L, [$]
  Start -> L = . R, [$]

I9:
  Start -> L = R ., [$]

I10:
  L -> . * R, [$]
  L -> . id, [$]
  L -> * . R, [$]
  R -> . L, [$]

I11:
  L -> id ., [$]

I12:
  R -> L ., [$]

I13:
  L -> * R ., [$]
```

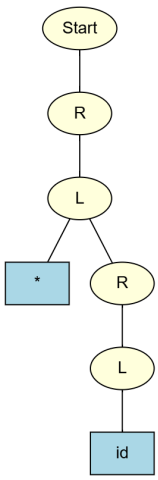
--- LR(1) Parsing Table ---

State	$	*	=	id	L	R	Start
0	-	s3	-	s4	5	1	2
1	r(Start -> R)[$]	-	-	-	-	-	-
2	accept	-	-	-	-	-	-
3	-	s3	-	s4	7	6	-
4	r(L -> id)[$]	-	r(L -> id)[=]	-	-	-	-
5	r(R -> L)[$]	-	s8	-	-	-	-
6	r(L -> * R)[$]	-	r(L -> * R)[=]	-	-	-	-
7	r(R -> L)[$]	-	r(R -> L)[=]	-	-	-	-
8	-	s10	-	s11	12	9	-
9	r(Start -> L = R)[$]	-	-	-	-	-	-
10	-	s10	-	s11	12	13	-
11	r(L -> id)[$]	-	-	-	-	-	-
12	r(R -> L)[$]	-	-	-	-	-	-
13	r(L -> * R)[$]	-	-	-	-	-	-

Parser Type: LR(1)
Is LR(1) Parseable: YES
Conflicts Found: 0

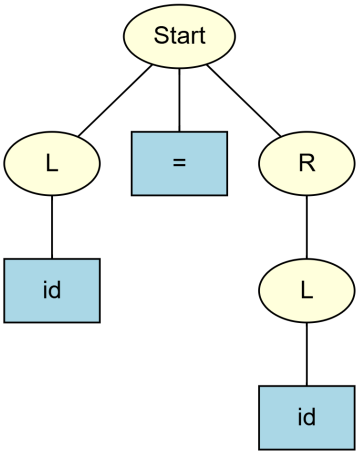
```
Input 1: * id
LR(1) Parsing:
SLR Parsing Trace:
Step     Stack                          Input                Action
--------------------------------------------------------------------------------
1        $0*3                           id $                 shift 3
2        $0*3id4                        $                    shift 4
3        $0*3L7                         $                    reduce L -> id
4        $0*3R6                         $                    reduce R -> L
5        $0L5                           $                    reduce L -> * R
6        $0R1                           $                    reduce R -> L
7        $0Start2                       $                    reduce Start -> R
8        $0Start2                       $                    accept

LR(1) Parsing result: ACCEPTED
```



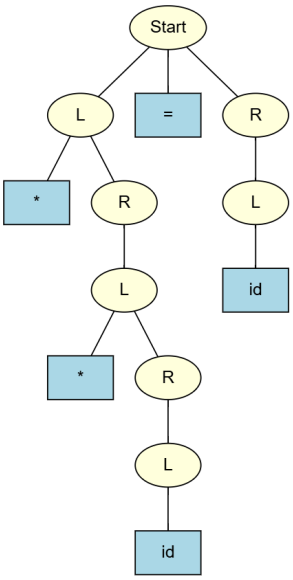

```
Input 2: id = id
LR(1) Parsing:
SLR Parsing Trace:
Step     Stack                          Input                Action
--------------------------------------------------------------------------------
1        $0id4                          = id $               shift 4
2        $0L5                           = id $               reduce L -> id
3        $0L5=8                         id $                 shift 8
4        $0L5=8id11                     $                    shift 11
5        $0L5=8L12                      $                    reduce L -> id
6        $0L5=8R9                       $                    reduce R -> L
7        $0Start2                       $                    reduce Start -> L = R
8        $0Start2                       $                    accept

LR(1) Parsing result: ACCEPTED
```




```
Input 3: * * id = id
LR(1) Parsing:
SLR Parsing Trace:
Step     Stack                          Input                Action
--------------------------------------------------------------------------------
1        $0*3                           * id = id $          shift 3
2        $0*3*3                         id = id $            shift 3
3        $0*3*3id4                      = id $               shift 4
4        $0*3*3L7                       = id $               reduce L -> id
5        $0*3*3R6                       = id $               reduce R -> L
6        $0*3L7                         = id $               reduce L -> * R
7        $0*3R6                         = id $               reduce R -> L
8        $0L5                           = id $               reduce L -> * R
9        $0L5=8                         id $                 shift 8
10       $0L5=8id11                     $                    shift 11
11       $0L5=8L12                      $                    reduce L -> id
12       $0L5=8R9                       $                    reduce R -> L
13       $0Start2                       $                    reduce Start -> L = R
14       $0Start2                       $                    accept

LR(1) Parsing result: ACCEPTED
```



```
Input 1: id = = id
LR(1) Parsing:
SLR Parsing Trace:
Step     Stack                          Input                Action
--------------------------------------------------------------------------------
1        $0id4                          = = id $             shift 4
2        $0L5                           = = id $             reduce L -> id
3        $0L5=8                         = id $               shift 8
4        $0L5=8                         = id $               ERROR: no action for (8, =)

LR(1) Parsing result: REJECTED


Input 2: * = id
LR(1) Parsing:
SLR Parsing Trace:
Step     Stack                          Input                Action
--------------------------------------------------------------------------------
1        $0*3                           = id $               shift 3
2        $0*3                           = id $               ERROR: no action for (3, =)

LR(1) Parsing result: REJECTED
```

7. Conclusion

Implementing both SLR(1) and LR(1) parsers from scratch provided deep insight into how compilers work. The shift-reduce parsing algorithm is elegant in its simplicity - a single loop with table lookups drives the entire parsing process. The complexity lies in building the tables themselves, which requires careful computation of items, closures, and lookaheads.

The LR(1) parser, while more complex, demonstrates how compilers handle the full complexity of programming language syntax. Understanding these concepts is essential for anyone interested in compiler design, language implementation, or formal language theory.

The successful implementation of both parsers, with proper conflict detection and parse tree generation, achieves all the project objectives and provides a solid foundation for understanding LR parsing techniques.